

Unit 2: Network Environment and Storage Structures

Configuring Network Environment

- ☞ After installing Oracle Database, you have a fully functional database with a client/server network environment that has been minimally configured.
- ☞ You need to configure your client/server network.

Overview of Network Configuration

- ☑ A **client** is any application that connects to Oracle Database to send or retrieve data. An Oracle Database client application can reside on any computer that has Oracle Database client software installed.
- ☑ **Oracle Net** is a software layer that resides on the client computer and on the Oracle Database host computer. It establishes and maintains the connection between the client application and the database over a network, and exchanges messages between them using industry standard protocols.
- ☑ For a client application and a database **to communicate**, the **client application must be able to identify the database it wants to connect to**, and the **database must provide an identification**. You can use a **service name** to connect to a database.
 - A **service name** is a logical representation of a database, which is the way a database is presented to clients.
 - A single database can be presented as multiple services.
- ☑ Service names can provide *location transparency* so that the client application does not have to know the server's location.
 - If the database is moved to another location, then you must reconfigure only Oracle Net. No changes are necessary to client applications.

Oracle Net Listener Configuration and Management

- ☑ On the database host, the **Oracle Net listener (the listener)**, is a process that **listens for client connection requests**. It receives incoming client connection requests and manages the traffic of these requests to the database server.
- ☑ The *default listener configuration file* is called **listener.ora**, and it is located in the *network/admin* subdirectory of the Oracle home directory.

- ☞ For example, if your Oracle home directory is `/u01/app/oracle/product/11.2.0/dbhome_1`, then the **listener.ora** file is created by default in the `/u01/app/oracle/product/11.2.0/dbhome_1/network/admin` directory.
- ☞ The file contains a **protocol address** that identifies the database. This address defines the **protocol the listener is listening** on and any other **protocol-specific information**.
- ☞ For example, the listener could be configured to listen at the following protocol address:

```
(DESCRIPTION=
  (ADDRESS=(PROTOCOL=tcp)(HOST=dbhost.example.com)
  (PORT=1521)))
```

This example shows a TCP/IP protocol address that specifies the host computer of the listener and a port number. The listener can listen for connection requests on network interfaces with either IP version 4 (IPv4) or IP version 6 (IPv6) addresses.

- ☑ The **listener.ora** file is automatically configured during installation.
 - Because the configuration parameters have default values, you can start and use a listener without configuring it. This default listener is named **LISTENER**, supports no service names on startup, and listens on the following TCP/IP protocol address

```
(ADDRESS=(PROTOCOL=tcp)(HOST=host_name)(PORT=1521))
```
- ☑ An Oracle database registers with the listener within a minute or so of starting up. The service names, or the databases that they represent, to which the listener forwards client requests, can be configured in the **listener.ora** file. This information can also be dynamically registered with the listener. Dynamic registration of services and databases with the listener is called **service registration**.
- ☑ Service registration is performed by the listener registration (LREG) process—an instance background process—of each database instance. Dynamic service registration does not require modification of the listener.ora file.

Client Connections

Followings are the examples some elements involved in client connections to the database.

- Connect Descriptors
- Connection Requests
- Naming Methods

Connect Descriptors

- The client uses a connect descriptor **to specify the database to which it wants to connect**. This connect descriptor contains a **protocol and a database service name**. A database can have multiple service names defined, so a specific service name must be specified for the connect descriptor. In a preconfigured database, there is only one service name, which defaults to the global database name.
- The following example shows a connect descriptor that enables clients to connect to a database with service name mydb.us.example.com:

DESCRIPTION=

(ADDRESS=(PROTOCOL=tcp)(HOST=my-server)(PORT=1521))

(CONNECT_DATA=

(SERVICE_NAME=mydb.us.example.com))

Connection Requests

- Users initiate a connection request by providing a **connect string**. A connect string **includes a user name and password, and a connect identifier**. This connect identifier can be the connect descriptor itself, or a name that resolves to the connect descriptor using mapping information stored in one or more repositories accessed with the naming methods described in "**Naming Methods**". This name is referred to as a **net service name**.

Naming Methods

- A naming method is a **resolution method used by a client application to resolve a connect identifier to a connect descriptor when attempting to connect to a database service**.

- Oracle Net provides support for the following naming methods:

1. **Easy Connect Naming**

The easy connect naming method enables clients **to connect to an Oracle database by using only a TCP/IP connect string consisting of a host name and service name**. The easy connect naming method **requires no configuration**.

2. **Local Naming**

The local naming method stores **connect descriptors, identified by their net service names**, in a **client configuration file** named **tnsnames.ora**. This file is located in the *ORACLE_HOME/network/admin* directory. When you create a database using Oracle Database Configuration Assistant (DBCA), **local naming is configured automatically**. You must then use the Net Configuration Assistant to create connect descriptors and their corresponding net service names.

3. **Directory Naming**

Directory naming resolves a **database service, net service name, or net service alias to a connect descriptor** stored in an **LDAP** (Lightweight Directory Application Protocol)-**compliant directory server**.

Tools for Configuring and Managing the Oracle Network

- Oracle Database provides tools to enable you to manage your network configuration. For example:
 - ☞ Net Configuration Assistant
 - ☞ Oracle Net Manager

Net Configuration Assistant

- During a typical database installation, Net Configuration Assistant automatically configures a listener called LISTENER that has a TCP/IP listening protocol address for the database. If you do a custom installation, then Net Configuration Assistant prompts you to configure a listener name and protocol address of your choice.
- Use Net Configuration Assistant for initial network configuration after database installation. Thereafter, you can use Oracle Net Manager to configure and administer your networks.

Oracle Net Manager

- Oracle Net Manager provides various network configuration features, including the ability to configure profiles

View Listener Configuration

- The Oracle Net listener (the listener) runs on your database host and handles incoming client requests. You can view the listener status at the command line

To view information about the listener at the command line:

1. Open a command window.
2. Configuring the Operating System Environment Variables
(<https://docs.oracle.com/en/database/oracle/oracle-database/19/admqslg/getting-started-with-database-administration.html#GUID-EC18C4A6-3BA5-4C14-9D76-B0DD62FEFFF2>)
3. Enter the following command:

```
lsnrctl status
```

Starting and Stopping the Listener

- The Oracle listener is set to start automatically whenever the host is restarted. However, when your system encounters unforeseen circumstances, or when you have manually stopped the listener, you can restart it at the command line.
- To start or stop the listener at the command line:
- Open a command window.
- Follow the steps listed in "[Configuring the Operating System Environment Variables](#)."
- Enter either of the following commands, depending on whether you want to start or stop the listener:

```
lsnrctl start
```

```
lsnrctl stop
```

Use TNSPING to Test Oracle Net Connectivity

- ☞ After you have configured the network, you should connect and test each component to ensure that the network is functioning properly. Oracle Net Services provides tools to help you test the listener, database, and Oracle Connection Manager.
- ☞ The TNSPING utility determines whether the listener for a service on an Oracle Net network can be reached successfully.
- ☞ If you can connect successfully from a client to a server (or a server to another server) using the TNSPING utility, then it displays an estimate of the round trip time (in milliseconds) it takes to establish a SQLNET connection to the Oracle listener corresponding to the server.
 - If it fails, then it displays a message describing the error that occurred. This enables you to see the network error that is occurring without the overhead of a database connection.
- ☞ Use the following command to test connectivity:


```
tnsping net_service_name count
```
- ☞ In the preceding command, the following arguments are used:
 - *net_service_name* must exist in *tnsnames.ora* file or the name service in use, such as NIS.
 - *count* determines how many times the program attempts to reach the server. This argument is optional.
- ☞ If the network service name specified is a database name, then TNSPING attempts to contact the corresponding listener. It does not actually determine whether the database is running. Use SQL*Plus to attempt a connection to the database.

Configure Net Services with Enterprise Manager

Here's a breakdown of the steps to configure Net Services with Enterprise Manager (EM) in Oracle 19c:

1. Access Enterprise Manager:

- Open a web browser and navigate to the Enterprise Manager Console URL provided by your administrator.
- Enter your login credentials and log in.

2. Navigate to the Target:

- In the navigation pane, locate and expand the **Database** section.
- Find the specific **database instance** you want to manage Net Services for.

3. Manage Net Services:

- Right-click on the database name and select **Net Services Administration** from the context menu. This opens the "Net Services Administration" page.

4. Configure Net Services:

- The "Net Services Administration" page provides various options for managing Net Services:

Listener Configuration:

- View and modify listener configuration details like listener name, host, port, and protocol stack.
- Add or remove listeners.
- Start, stop, or restart the listener service.

To configure the listener:

- Click on the **Listener** tab.
- You can view existing listener details or click **Create** to define a new listener.
- Modify the desired settings, including:
 - **Listener Name:** Unique identifier for the listener.
 - **Host:** Hostname or IP address where the listener runs.
 - **Port:** Port number used by the listener for communication.
 - **Protocol Stack:** Communication protocol used by the listener (e.g., TCP/IP).
- Click **Save** to apply the changes.

TNS Names Configuration:

- View and modify existing TNS aliases and their connection details.

- Create new TNS aliases for connecting to different databases using specific connection parameters.
- Set the default TNS alias for the database.

To configure TNS names:

- Click on the **TNS Names** tab.
- You can view existing aliases or click **Create** to define a new one.
- Specify the following details:
 - **Alias Name:** User-friendly name for the connection.
 - **Description:** Optional description of the connection.
 - **Connection Details:** Define the connection parameters like hostname, port, service name, and other relevant settings.
- ☒ Click **Save** to apply the changes.

Database Services:

- View and manage database services registered with the listener.
- Register new database services to allow applications to connect using service names.
- Modify the configuration of existing database services.

To configure database services:

- Click on the **Database Services** tab.
- You can view existing services or click **Create** to define a new one.
- Specify the following details:
 - **Service Name:** Unique identifier for the service within the listener.
 - **SID:** System Identifier of the database instance.
 - **Database Host:** Hostname or IP address of the database server.
- Click **Save** to apply the changes.

5. Review and Apply:

- After making desired changes, ensure you review any validation messages or errors displayed by EM.
- Click **Save and Apply** to commit the configuration changes.

Additional Considerations:

- Remember to restart the listener service after making changes to its configuration for the changes to take effect.
- EM might require specific privileges or roles for certain functionalities like creating or modifying resources. Consult your administrator for access control details.
- While EM offers a user-friendly interface, ensure you understand the implications of any changes you make to your Net Services configuration, as it can impact application connectivity and database access.

Oracle Storage Structure

- Basic structural architecture of the Oracle database includes logical and physical storage structures.
- The **physical database structures** of an Oracle database are viewable at the operating system level.
- **Logical Storage Structures** are created and recognized by Oracle Database and are not known to the operating system.

Overview of Data Storage

Mechanisms for Storing Database Files

Several mechanisms are available for allocating and managing the storage of these files.

Mechanisms include:

☒ **Operating system file system**

Most Oracle databases store files in a file system, which is a structure for storing and retrieving data.

A file system enables storage space to be used for application data in a standard and consistent way. Each file has a name and appears as a contiguous address space to applications such as Oracle Database. The database can create, read, write, resize, and delete files.

A file system is commonly built on top of a logical volume constructed by a software package called a logical volume manager (LVM). The LVM enables pieces of multiple physical disks to combine into what appears as a single contiguous address space.

☑ **Cluster file system**

A **cluster file system** is a distributed file system that is a cluster of servers that collaborate to provide consistency and high performance to their clients. In an Oracle RAC environment, a cluster file system makes shared storage appear as a file system shared by many computers in a clustered environment. With a cluster file system, the failure of a computer in the cluster does not make the file system unavailable. In an operating system file system, however, if a computer sharing files through NFS or other means fails, then the file system is unavailable.

☑ **Oracle Persistent Memory Filestore (PMEM Filestore)**

PMEM Filestore is designed to store database files in Persistent Memory (PMEM), which is byte-addressable persistent memory. You create a filestore using the `CREATE PMEM FILESTORE` command. At the Linux operating system level, the filestore stores data in a large `ext4` file system mounted in `fsdax` mode. PMEM Filestore appears to Oracle Database as a native file system on Linux.

☑ **Oracle Automatic Storage Management (Oracle ASM)**

Oracle ASM includes a file system designed exclusively for use by Oracle Database.

Oracle Managed Files and User-Managed Files

- **Oracle Managed Files** is a file naming strategy that enables you to specify operations in terms of database objects rather than file names. For example, you can create a tablespace without specifying the names of its data files.

- Oracle Managed Files eliminates the need for administrators to directly manage the operating system files in a database. Oracle ASM requires Oracle Managed Files.
- With **user-managed files**, you directly manage the operating system files in the database. You make the decisions regarding file structure and naming. For example, when you create a tablespace you set the name and path of the tablespace data files.
- Through initialization parameters, you specify the file system directory for a specific type of file. The Oracle Managed Files feature ensures that the database creates a unique file and deletes it when no longer needed. The database internally uses standard file system interfaces to create and delete files for data files and temp files, control files, and recovery-related files stored in the fast recovery area.
- Oracle Managed Files does not eliminate existing functionality. You can create new files while manually administering old files. Thus, a database can have a mixture of Oracle Managed Files and user-managed files.

Data block

- A data block is the smallest logical unit of data storage in Oracle Database. One logical data block corresponds to a specific number of bytes of physical disk space, for example, 2 KB. Data blocks are the smallest units of storage that Oracle Database can use or allocate

Database Block Size

- Every database has a database block size.
- The `DB_BLOCK_SIZE` initialization parameter sets the data block size for a database when it is created. The size is set for the SYSTEM and SYSAUX tablespaces and is the default for all other tablespaces. The database block size cannot be changed except by re-creating the database.
- If `DB_BLOCK_SIZE` is not set, then the default data block size is operating system-specific. The standard data block size for a database is 4 KB or 8 KB. If the size differs for data blocks and operating system blocks, then the data block size must be a multiple of the operating system block size.

Overview of Tablespaces

- A **tablespace** is a logical storage container for segments. Segments are database objects, such as tables and indexes, that consume storage space. At the physical level, a tablespace stores data in one or more data files or temp files.
- A database must have the SYSTEM and SYSAUX tablespaces. The following figure shows the tablespaces in a typical database. The following sections describe the tablespace types.

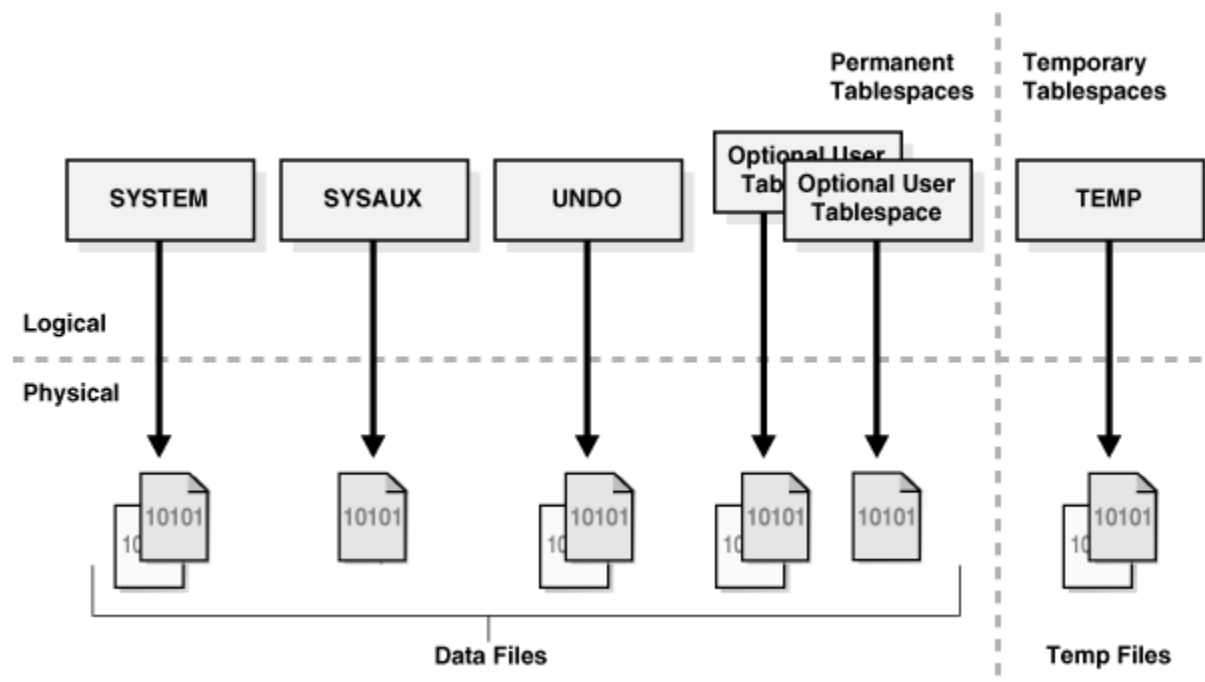


Figure 1: Tablespace

Permanent Tablespaces

A permanent tablespace groups persistent schema objects. The segments for objects in the tablespace are stored physically in data files.

Each database user is assigned a default permanent tablespace. A very small database may need only the default SYSTEM and SYSAUX tablespaces. However, Oracle recommends that you create at least one tablespace to store user and application data. You can use tablespaces to achieve the following goals:

- Control disk space allocation for database data
- Assign a quota (space allowance or limit) to a database user

- Take individual tablespaces online or offline without affecting the availability of the whole database
- Perform backup and recovery of individual tablespaces
- Import or export application data by using the Oracle Data Pump utility

The SYSTEM Tablespace

- The **SYSTEM** tablespace is a necessary administrative tablespace included with the database when it is created. Oracle Database uses SYSTEM to manage the database.
- The SYSTEM tablespace includes the following information, all owned by the SYS user:
 - The data dictionary
 - Tables and views that contain administrative information about the database
 - Compiled stored objects such as triggers, procedures, and packages
- The SYSTEM tablespace is managed as any other tablespace, but requires a higher level of privilege and is restricted in some ways. For example, you cannot rename or drop the SYSTEM tablespace.
- By default, Oracle Database sets all newly created user tablespaces to be locally managed. In a database with a locally managed SYSTEM tablespace, you cannot create dictionary-managed tablespaces (which are deprecated). However, if you execute the CREATE DATABASE statement manually and accept the defaults, then the SYSTEM tablespace is dictionary managed. You can migrate an existing dictionary-managed SYSTEM tablespace to a locally managed format.

The SYSAUX Tablespace

- The SYSAUX tablespace is an auxiliary tablespace to the SYSTEM tablespace.
- Because SYSAUX is the default tablespace for many Oracle Database features and products that previously required their own tablespaces, it reduces the number of tablespaces required by the database. It also reduces the load on the SYSTEM tablespace.
- Database creation or upgrade automatically creates the SYSAUX tablespace. During normal database operation, the database does not allow

the SYSAUX tablespace to be dropped or renamed. If the SYSAUX tablespace becomes unavailable, then core database functionality remains operational. The database features that use the SYSAUX tablespace could fail, or function with limited capability.

Undo Tablespaces

- An undo tablespace is a locally managed tablespace reserved for system-managed undo data.
- Like other permanent tablespaces, undo tablespaces contain data files. Undo blocks in these files are grouped in extents.
- A new installation of Oracle Database automatically creates an undo tablespace.
- A database can contain multiple undo tablespaces, but only one can be in use at a time.
- When an instance attempts to open a database, Oracle Database automatically selects the first available undo tablespace. If no undo tablespace is available, then the instance starts without an undo tablespace and stores undo data in the SYSTEM tablespace.
 - Storing undo data in SYSTEM is not recommended.

Shadow Tablespaces

- A shadow tablespace is a bigfile tablespace intended for shadow **lost write protection**.
- Shadow lost write protection provides fast detection and immediate response to a **lost write**.
- A data block lost write occurs when an I/O subsystem acknowledges the completion of the block write even though the write did not occur or when a former image of the block overwrites the current image.
- An undetected lost write can result in data corruption because the incorrect data can be used for other DML transactions. For example, a transaction can read old and incorrect data from one table, and then update hundreds of other tables based on this data. In this way, data corruption can spread throughout the database.

Shadow lost write protection provides the following benefits:

- ☞ It detects a lost write before it is consumed for standard DML, SQL*Loader conventional path load, direct path load, and RMAN backups.
- ☞ No standby database is necessary, as in the lost write protection introduced in Oracle Database 11g.
- ☞ You can enable shadow lost write protection for specific tablespaces and data files. You do not need to track all data.
- ☞ You can replace one shadow tablespace with another to change its configuration or location.
- ☞ You can suspend and resume shadow lost write protection for a tablespace or data file.
- ☞ You can enable or disable it for the entire non-CDB or PDB with a single ALTER DATABASE ... LOST WRITE TRACKING statement. Note that the PROP\$ table indicates whether tracking is enabled for a PDB.

Temporary Tablespaces

- A temporary tablespace contains transient data that persists only for the duration of a session. No permanent schema objects can reside in a temporary tablespace. A temp file stores temporary tablespace data.
- Temporary tablespaces can improve the concurrency of multiple sort operations that do not fit in memory. These tablespaces also improve the efficiency of space management operations during sorts.

Shared and Local Temporary Tablespaces

- Temporary tablespaces are either shared or local.
 - A shared temporary tablespace stores temp files on shared disk, so that the temporary space is accessible to all database instances. In contrast, a local temporary tablespace stores separate, non-shared temp files for every database instance. Local temporary tablespaces are useful for Oracle Real Application Clusters or Oracle Flex Clusters.
 - You can create local temporary tablespaces for both read-only and read/write database instances. When many read-only instances access a single database, local temporary tablespaces can improve performance for queries that involve sorts, hash aggregations, and joins. The advantages are:

- Improving I/O performance by using local rather than shared disk storage
- Avoiding expensive cross-instance temporary space management
- Improving instance startup performance by eliminating on-disk space metadata management

Default Temporary Tablespaces

- Every database user account is assigned a default shared temporary tablespace. If the database contains local temporary tablespaces, then every user account is also assigned default local temporary storage.
- You can specify a different temporary tablespace for a user account with the CREATE USER or ALTER USER statements. Oracle Database use the system-level default temporary tablespace for users for whom you do not specify a different temporary tablespace.

Overview of Data Files

- At the operating system level, Oracle Database stores data in structures called data files. Every Oracle database, whether a CDB or PDB, must have at least one data file.

Use of Data Files

- Oracle Database physically stores tablespace data in data files.
- Each nonpartitioned schema object and each partition of an object is stored in its own segment, which belongs to only one tablespace. For example, the data for a nonpartitioned table is stored in a single segment, which in turn is stored in one tablespace.
- Tablespaces and data files are closely related, but have important differences:
 - ☞ Each tablespace consists of one or more data files, which conform to the operating system in which Oracle Database is running.
 - ☞ The data for a database is collectively stored in the data files located in each tablespace of the database.
 - ☞ A segment can span one or more data files, but it cannot span multiple tablespaces.

- ☞ A CDB must have the SYSTEM and SYSAUX tablespaces. Each PDB within a CDB has its own SYSTEM and SYSAUX tablespaces. Oracle Database automatically allocates the first data files of any database for the SYSTEM tablespace during database creation.
- The SYSTEM tablespace contains the data dictionary, a set of tables that contains database metadata. Typically, a database also has an undo tablespace and a temporary tablespace (usually named TEMP). The following figure shows the relationship between tablespaces, data files, and segment

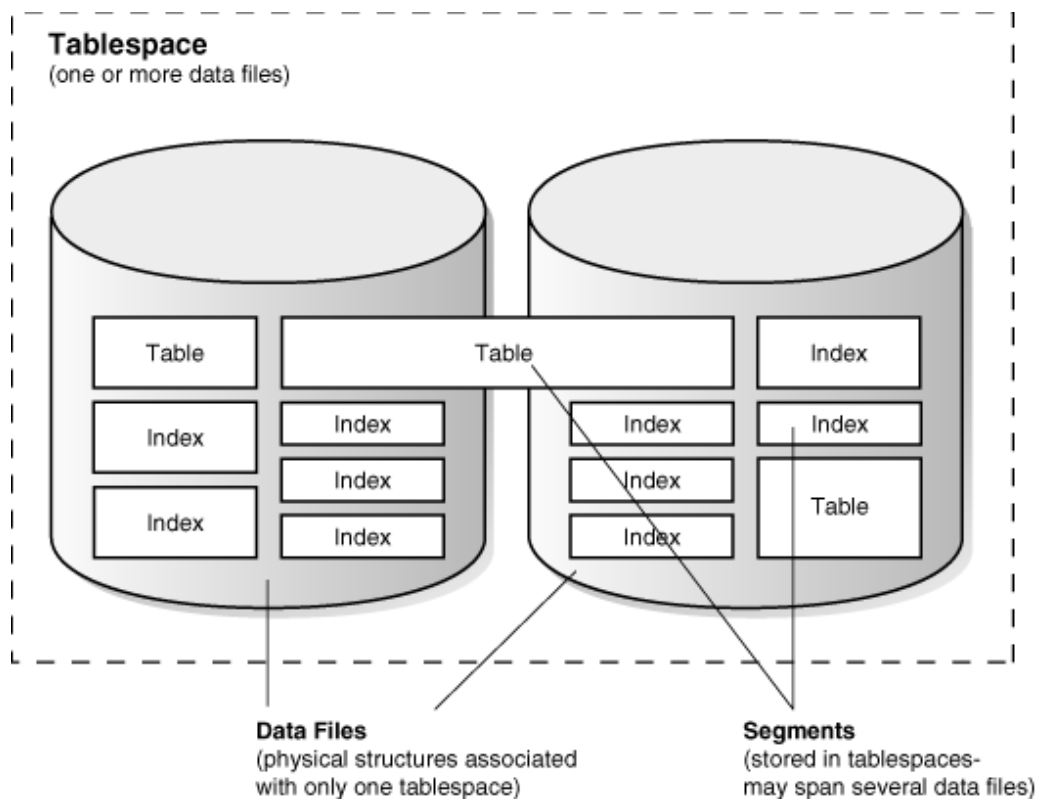


Figure 2: Data Files and Tablespaces

Permanent and Temporary Data Files

- A **permanent tablespace** contains persistent schema objects. Objects in permanent tablespaces are stored in data files.
- A **temporary tablespace** contains schema objects only for the duration of a session. Locally managed temporary tablespaces have temporary files (temp files), which are special files designed to store data in hash, sort, and other operations. Temp files also store result set data when insufficient space exists in memory.
- Temp files are similar to permanent data files, with the following exceptions:

- Permanent database objects such as tables are never stored in temp files.
- Temp files are always set to NOLOGGING mode, which means that they never have redo generated for them. Media recovery does not recognize temp files.
- You cannot make a temp file read-only.
- You cannot create a temp file with the ALTER DATABASE statement.
- When you create or resize temp files, they are not always guaranteed allocation of disk space for the file size specified. On file systems such as Linux and UNIX, temp files are created as sparse files. In this case, disk blocks are allocated not at file creation or resizing, but as the blocks are accessed for the first time.
- Temp file information is shown in the data dictionary view DBA_TEMP_FILES and the dynamic performance view V\$TEMPFILE, but not in DBA_DATA_FILES or the V\$DATAFILE view

Online and Offline Data Files

- Every data file is either online (available) or offline (unavailable).
- You can alter the availability of individual data files or temp files by taking them offline or bringing them online. The database cannot access offline data files until they are brought online.
- You may take data files offline for many reasons, including performing offline backups or block corruption. The database takes a data file offline automatically if the database cannot write to it.
- Like a data file, a tablespace itself is offline or online. When you take a data file offline in an online tablespace, the tablespace itself remains online. You can make all data files of a tablespace temporarily unavailable by taking the tablespace itself offline.
- Starting in Oracle Database 12c, you can use the ALTER DATABASE MOVE DATAFILE statement to move an online data file from one physical file to another while the database is open and accessing the file. You can use this technique to achieve the following goals:
 - Move a tablespace from one kind of storage to another
 - Move data files that are accessed infrequently to lower cost storage

- Make a tablespace read-only and move its data files to write-once storage, such as a write once read many (WORM) drive
- Move a database into Oracle ASM

Data Files Data Dictionary Views

A set of data dictionary views provides useful information about the data files of a database.

View	Description
<u>DBA_DATA_FILES</u>	Provides descriptive information about each data file, including the tablespace to which it belongs and the file ID. The file ID can be used to join with other views for detail information.
<u>DBA_EXTENTS</u> <u>USER_EXTENTS</u>	DBA view describes the extents comprising all segments in the database. Contains the file ID of the data file containing the extent. USER view describes extents of the segments belonging to objects owned by the current user.
<u>DBA_FREE_SPACE</u> <u>USER_FREE_SPACE</u>	DBA view lists the free extents in all tablespaces. Includes the file ID of the data file containing the extent. USER view lists the free extents in the tablespaces accessible to the current user.
<u>V\$DATAFILE</u>	Contains data file information from the control file
<u>V\$DATAFILE_HEADER</u>	Contains information from data file headers

This example illustrates the use of one of these views, V\$DATAFILE.

```
SELECT NAME,
       FILE#,
       STATUS,
       CHECKPOINT_CHANGE# "CHECKPOINT"
FROM   V$DATAFILE;
```

NAME	FILE#	STATUS	CHECKPOINT
-----	----	-----	-----
/u01/oracle/rbdb1/system01.dbf	1	SYSTEM	3839
/u02/oracle/rbdb1/temp01.dbf	2	ONLINE	3782
/u02/oracle/rbdb1/users03.dbf	3	OFFLINE	3782

FILE# lists the file number of each data file; the first data file in the SYSTEM tablespace created with the database is always file 1. STATUS lists other information about a data file. If a data file is part of the SYSTEM tablespace, its status is SYSTEM (unless it requires recovery). If a data file in a non-SYSTEM tablespace is online, its status is ONLINE. If a data file in a non-SYSTEM tablespace is offline, its status can be either OFFLINE or RECOVER. CHECKPOINT lists the final SCN (system change number) written for the most recent checkpoint of a data file.

Create and Alter Tablespace Commands

Basic syntax of CREATE TABLESPACE to create a tablespace

CREATE TABLESPACE tablespace_name

DATAFILE 'datafile_path'

SIZE [size_tablespace];

```
CREATE TABLESPACE bipin_tbs1 DATAFILE 'bipin_df1.dbf' SIZE 4M;
```

- This creates permanent tablespace.
- Use the ALTER TABLESPACE statement to alter an existing tablespace or one or more of its data files.
- To change the status to offline

```
ALTER tablespace bipin_tbs1 OFFLINE;
```

- To change the status to online

```
ALTER tablespace bipin_tbs1 ONLINE;
```

- To add a data file

```
ALTER TABLESPACE bipin_tbs1 ADD DATAFILE 'bipin_df.dat'
SIZE 3M;
```

- To rename the tablespace

```
ALTER TABLESPACE old_tablespace_name RENAME TO
new_tablespace_name;
```

Create and Manage Datafiles

- CREATE TABLESPACE statement **creates datafiles** specified.
- ALTER TABLESPACE statement can be used to modify attributes of an existing tablespace, such as **adding datafiles**, **resizing datafiles**, or changing storage parameters.
 - You can add the AUTOEXTEND ON MAXSIZE size clause to enable autoextension for the datafile. This allows the datafile to grow automatically when it reaches its initial size limit.
- The **ALTER DATABASE DATAFILE** statement is used for specific operations on individual datafiles, such as renaming or relocating a datafile.
- **Resizing Using ALTER DATABASE:**

```
ALTER DATABASE DATAFILE 'file_name' RESIZE new_size;
```

Example: ALTER DATABASE DATAFILE 'bipin_df3.dat' RESIZE 200M

- To rename the datafile use following statement

```
ALTER DATABASE RENAME FILE 'old_file_name' TO 'new_file_name';
```

Drop Tablespaces and Datafiles

- Use the **DROP TABLESPACE** statement to remove a tablespace from the database.
- When you drop a tablespace, Oracle Database does not place it in the recycle bin. Therefore, you cannot subsequently either purge or undrop the tablespace.

We can use the DROP TABLESPACE statement to permanently remove it from the database:

```
DROP TABLESPACE bipin_tbs1;
```

To drop the tbs_02 tablespace and deletes all associated operating system data files:

```
DROP TABLESPACE tbs_02 INCLUDING CONTENTS AND DATAFILES;
```

Use the **ALTER TABLESPACE DROP DATAFILE '<datafile_path>'** statement to drop the datafile from the tablespace. It involves dropping the datafile itself while keeping the tablespace.

Using Oracle Managed Files (OMF)

- Oracle Managed Files eases database administration, reduces errors, and reduces wasted disk space.
- Oracle Managed Files (OMF) is a feature that automates the management of database files on the operating system
- OMF takes care of creating and deleting database files like datafiles, redo log files, and control files. You specify the database object, and OMF handles the file system operations behind the scenes.
- OMF generates unique and standardized names for the database files. This simplifies administration and avoids naming conflicts.
- Using Oracle Managed Files simplifies the administration of an Oracle Database. Oracle Managed Files eliminates the need for you, the DBA, to directly manage the operating system files that comprise an Oracle Database.
 - With Oracle Managed Files, you specify file system directories in which the database automatically creates, names, and manages files at the database object level. For example, you need only specify that you want to create a tablespace; you do not need to specify the name and path of the tablespace's data file with the DATAFILE clause. This feature works well with a logical volume manager (LVM).
 - The database internally uses standard file system interfaces to create and delete files as needed for the following database structures:
 - Tablespaces
 - Redo log files
 - Control files
 - Archived logs
 - Block change tracking files
 - Flashback logs
 - RMAN backups
 - Through initialization parameters, you specify the file system directory to be used for a particular type of file. The database then ensures that a unique file, an Oracle managed file, is created and deleted when no longer needed.
 - This feature does not affect the creation or naming of administrative files such as trace files, audit files, alert logs, and core files.

- Oracle Managed Files are most useful for the following types of databases:
 - Databases that are supported by the following:
 - A logical volume manager that supports striping/RAID and dynamically extensible logical volumes
 - A file system that provides large, extensible files
 - Low end or test databases

What Is a Logical Volume Manager?

A logical volume manager (LVM) is a software package available with most operating systems. Sometimes it is called a logical disk manager (LDM). It allows pieces of multiple physical disks to be combined into a single contiguous address space that appears as one disk to higher layers of software.

An LVM can make the logical volume have better capacity, performance, reliability, and availability characteristics than any of the underlying physical disks. It uses techniques such as mirroring, striping, concatenation, and RAID 5 to implement these characteristics.

Some LVMs allow the characteristics of a logical volume to be changed after it is created, even while it is in use. The volume may be resized or mirrored, or it may be relocated to different physical disks.

Benefits of Using Oracle Managed Files

Oracle Managed Files provides several benefits. Consider the following benefits of using Oracle Managed Files:

- They make the administration of the database easier.
- There is no need to invent file names and define specific storage requirements. A consistent set of rules is used to name all relevant files. The file system defines the characteristics of the storage and the pool where it is allocated.
- They reduce corruption caused by administrators specifying the wrong file.
- Each Oracle managed file and file name is unique. Using the same file in two different databases is a common mistake that can cause very large down times and

loss of committed transactions. Using two different names that refer to the same file is another mistake that causes major corruptions.

- They reduce wasted disk space consumed by obsolete files.
- Oracle Database automatically removes old Oracle Managed Files when they are no longer needed. Much disk space is wasted in large systems simply because no one is sure if a particular file is still required. This also simplifies the administrative task of removing files that are no longer required on disk and prevents the mistake of deleting the wrong file.
- They simplify creation of test and development databases.
- You can minimize the time spent making decisions regarding file structure and naming, and you have fewer file management tasks. You can focus better on meeting the actual requirements of your test or development database.
- Oracle Managed Files make development of portable third-party tools easier.
- Oracle Managed Files eliminate the need to put operating system specific file names in SQL scripts.

Enabling the Creation and Use of Oracle Managed Files

- You set certain initialization parameters to enable and use Oracle Managed Files.
- The following table lists the initialization parameters that enable the use of Oracle Managed Files.

Initialization Parameter	Description
DB_CREATE_FILE_DEST	Defines the location of the default file system directory or Oracle ASM disk group where the database creates data files or temp files when no file specification is given in the create operation. Also used as the default location for redo log and control files if DB_CREATE_ONLINE_LOG_DEST_ <i>n</i> are not specified.
DB_CREATE_ONLINE_LOG_DEST_ <i>n</i>	Defines the location of the default file system directory or Oracle ASM disk group for redo log files and control file creation when no file specification is given in the create operation. By changing <i>n</i> , you can use this initialization parameter multiple times, where <i>n</i> specifies a multiplexed copy of the redo log or control file. You can specify up to five multiplexed copies.

DB_RECOVERY_FILE_DEST	Defines the location of the Fast Recovery Area, which is the default file system directory or Oracle ASM disk group where the database creates RMAN backups when no format option is used, archived logs when no other local destination is configured, and flashback logs. Also used as the default location for redo log and control files or multiplexed copies of redo log and control files if DB_CREATE_ONLINE_LOG_DEST_n are not specified. When this parameter is specified, the DB_RECOVERY_FILE_DEST_SIZE initialization parameter must also be specified.
------------------------------	--

- The file system directories specified by these parameters must already exist; the database does not create them. The directory must also have permissions to allow the database to create the files in it.
- The default location is used whenever a location is not explicitly specified for the operation creating the file. The database creates the file name, and a file thus created is an Oracle managed file.
- Both of these initialization parameters are dynamic, and can be set using the ALTER SYSTEM or ALTER SESSION statement.

Creating Oracle Managed Files

- You can use Oracle Managed Files to create data files, temp files, control files, redo log files, and archived log. If you have met any of the following conditions, then Oracle Database creates Oracle Managed Files for you, as appropriate, when no file specification is given in the create operation:
 - You have included any of the DB_CREATE_FILE_DEST, DB_RECOVERY_FILE_DEST, or DB_CREATE_ONLINE_LOG_DEST_n initialization parameters in your initialization parameter file.
 - You have issued the ALTER SYSTEM statement to dynamically set any of DB_RECOVERY_FILE_DEST, DB_CREATE_FILE_DEST, or DB_CREATE_ONLINE_LOG_DEST_n initialization parameters
 - You have issued the ALTER SESSION statement to dynamically set any of

the DB_CREATE_FILE_DEST, DB_RECOVERY_FILE_DEST, or DB_CREATE_ONLINE_LOG_DEST_n initialization parameters

- If a statement that creates an Oracle managed file finds an error or does not complete due to some failure, then any Oracle Managed Files created by the statement are automatically deleted as part of the recovery of the error or failure. However, because of the large number of potential errors that can occur with file systems and storage subsystems, there can be situations where you must manually remove the files using operating system commands.

How Oracle Managed Files Are Named?

- ❖ The file names of Oracle Managed Files comply with the Optimal Flexible Architecture (OFA) standard for file naming.
- ❖ The assigned names are intended to meet the following requirements:
 - Database files are easily distinguishable from all other files.
 - Files of one database type are easily distinguishable from other database types.
 - Files are clearly associated with important attributes specific to the file type. For example, a data file name may include the tablespace name to allow for easy association of data file to tablespace, or an archived log name may include the thread, sequence, and creation date.
- ❖ No two Oracle Managed Files are given the same name. The name that is used for creation of an Oracle managed file is constructed from three sources:
 - The default creation location
 - A file name template that is chosen based on the type of the file. The template also depends on the operating system platform and whether or not Oracle Automatic Storage Management is used.
 - A unique string created by Oracle Database or the operating system. This ensures that file creation does not damage an existing file and that the file cannot be mistaken for some other file.

Creating Oracle Managed Files at Database Creation

- ❖ The CREATE DATABASE statement can perform actions related to Oracle Managed Files.

Specifying Control Files at Database Creation

- At database creation, the control file is created in the files specified by the CONTROL_FILES initialization parameter. If the CONTROL_FILES parameter is not set and at least one of the initialization parameters required for the creation of Oracle Managed Files is set, then an Oracle managed control file is created in the default control file destinations.

Specifying Redo Log Files at Database Creation

- The LOGFILE clause is not required in the CREATE DATABASE statement, and omitting it provides a simple means of creating Oracle managed redo log files.
- If the LOGFILE clause is omitted, then redo log files are created in the default redo log file destinations.

Specifying the SYSTEM and SYSAUX Tablespace Data Files at Database Creation

- The DATAFILE or SYSAUX DATAFILE clause is not required in the CREATE DATABASE statement, and omitting it provides a simple means of creating Oracle managed data files for the SYSTEM and SYSAUX tablespaces.

Specifying the Undo Tablespace Data File at Database Creation

- The DATAFILE subclause of the UNDO TABLESPACE clause is optional and a file name is not required in the file specification .
- If a file name is not supplied and the DB_CREATE_FILE_DEST parameter is set, then an Oracle managed data file is created in the DB_CREATE_FILE_DEST directory. If DB_CREATE_FILE_DEST is not set, then the statement fails with a syntax error.

Specifying the Default Temporary Tablespace Temp File at Database Creation

- The TEMPFILE subclause is optional for the DEFAULT TEMPORARY TABLESPACE clause and a file name is not required in the file specification.

- If a file name is not supplied and the DB_CREATE_FILE_DEST parameter set, then an Oracle managed temp file is created in the DB_CREATE_FILE_DEST directory. If DB_CREATE_FILE_DEST is not set, then the CREATE DATABASE statement fails with a syntax error.

CREATE DATABASE Statement Using Oracle Managed Files: Examples

Examples in given link illustrate creating a database with the CREATE DATABASE statement when using the Oracle Managed Files feature.

<https://docs.oracle.com/en/database/oracle/oracle-database/19/admin/using-oracle-managed-files.html#GUID-9E04D83B-4758-4E7D-B2A7-673B123F3989>

Visit the given link for detail.

Use Enterprise Manager to view the Storage Structure of the Database

Note: Starting with Oracle Database 19c, Oracle EM Express, the default management option for Oracle Database, is based on Java JET technology. Currently, only the Performance menu is available in the 19c JET version. Therefore, we should use 18c Flash Oracle EM. Use the following command to revert to Flash Oracle EM Express:

```
SQL> @?/rdbms/admin/execemx emx
```

To return to 19c JET Oracle EM Express, use the following command:

```
SQL> @?/rdbms/admin/execemx omx
```

(Since Flash is outdated most of the browsers do not support this)

An Oracle database is made up of physical and logical structures. Physical structures are defined in the operating system. Physical files that store data on a disk are an example of a physical structure. Logical structures are defined in the database and are

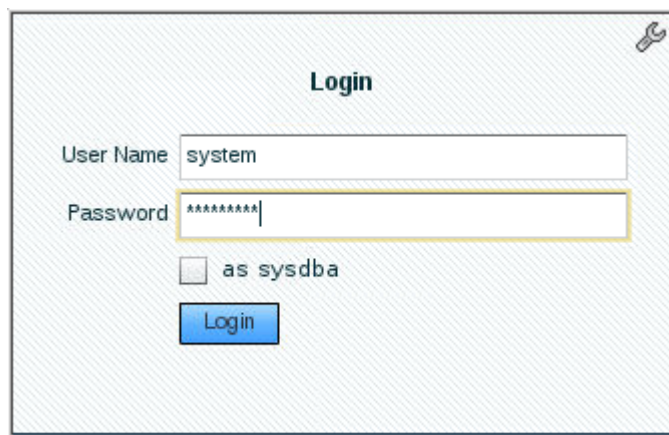
not known in the operating system. To view storage structure information, perform the following steps:

To View Tablespace and Data File Information

1. The pluggable database should be open for the Enterprise Management Database Express to work. If the pluggable database is not open, open the pluggable database and proceed to step 2.
2. Enter the URL for the Enterprise Manager Database Express for PDB in your browser and log in as the SYSTEM user.

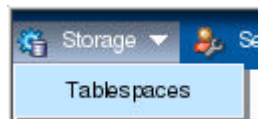
(Use: following command to know about the port number:

```
SQL> SELECT dbms_xdb_config.gethttpsport() from dual; )
```



This screenshot shows the Enterprise Manager login page for PDB instance. The User Name is set to "system" and the Password is entered. The "as sysdba" checkbox is unchecked and the Login button is being clicked.

3. Select Tablespaces in the Storage menu.



This screenshot shows the Storage Menu. The Storage Menu has just one link for Tablespaces which is being clicked.

4. The Tablespaces page is displayed.

This screenshot shows the Tablespaces page with the following tablespaces entry:

- SYSAUX
- SYSTEM
- TEMP

- UNDO...
- USERS

Name	Size	Free ...	Us...	Auto...	M	Status	Type	G..	Auto...	Dire...
⊞ SYS...	370MB	27MB	92.7	✓	Unl	●			✓	/scra...
⊞ SYS...	270MB	8MB	97	✓	Unl	●				/scra...
⊞ TEMP	62MB	62MB		✓	Unl	●				/scra...
⊞ UN...	100MB	40MB	60	✓	Unl	●				/scra...
⊞ USE...	5MB	4MB	20	✓	Unl	●			✓	/scra...

- Expand one of the tablespaces to view information about the datafiles allocated to the tablespace

Name	Size	Fre...	U...	Auto...	M	Status	Type	G	Auto...	Dir...
⊞ SYSAUX	370MB	27MB	92.7	✓	Un	●			✓	/scr...
sysaux01...	370MB	27MB	92.7	✓	Un	●				/scr...
⊞ SYSTEM	270MB	8MB	97	✓	Un	●				/scr...
⊞ TEMP	62MB	62MB		✓	Un	●				/scr...

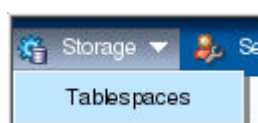
This screenshot shows the Tablespaces page with the SYSAUX tablespace expanded. When expanded, the SYSAUX tablespace shows 1 DBF file named sysaux01.dbf.

- Click ORCL to return to the Database Home page.

To Create a Tablespace

- ☞ A number of tablespaces are automatically created when you create the database. To create a new tablespace, perform the following steps

- Select Tablespaces in the Storage menu.



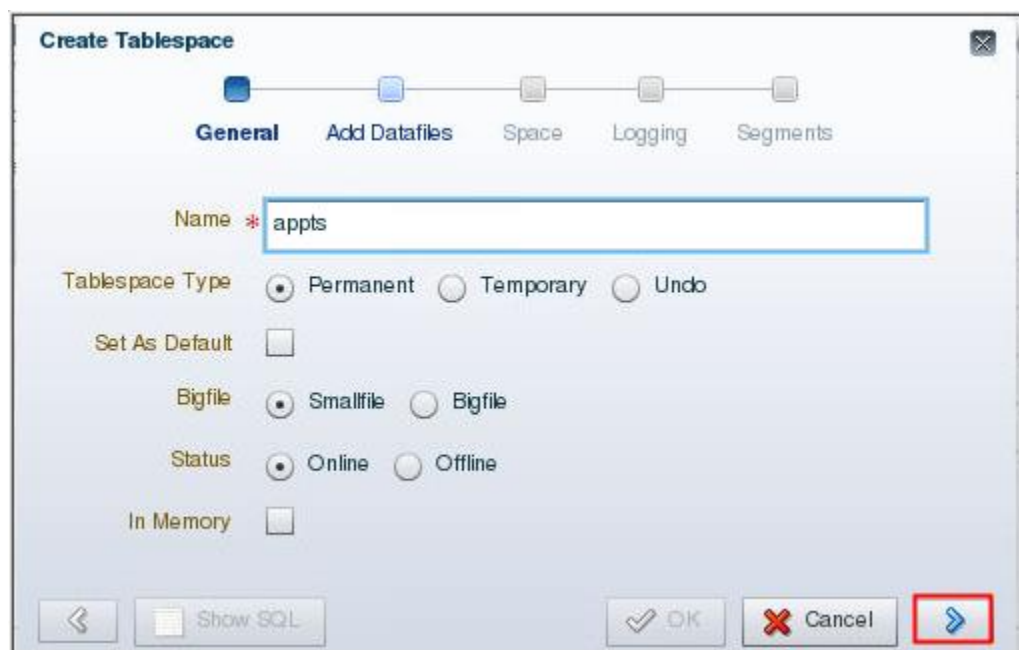
This screenshot shows the Storage Menu. The Storage Menu has just one link for Tablespaces which is being clicked.

2. On the Tablespaces page, click Create.



This screenshot shows the Tablespaces page. Create button is being clicked.

3. Enter a name for the new tablespace in the Name field. Accept the defaults for Tablespace type, Bigfile, and Status. Click the right arrow.



This screenshot shows the Create Tablespace wizard - Step 1 (General). The following values are set:

Name: appts

Tablespace Type: Permanent

Set As Default: (unchecked)

Big File: Smallfile

Status: Online

In Memory: (unchecked)

There are two active buttons - Cancel and Next. The Next button is being clicked.

4. Enter a file name in the Datafiles field and click the green plus symbol

Create Tablespace

General **Add Datafiles** Space Logging Segments

Using Oracle-Managed Files ☐ ⓘ

Datafiles * appts.dbf + X

File Name

File Size * 100M ⓘ

Reuse Existing File ☐

Auto Extend ☒ ⓘ

Increment 100M

Maximum File Size Unlimited

⏪ Show SQL ✓ OK ✗ Cancel ⏩

This screenshot shows the Create Tablespace wizard - Step 2 (Add Datafiles).
 "appts.dbf" is entered in the Datafiles field and the + (add) button is being clicked.
 x (delete) button
 File Size: 100M
 Reuse Existing File: (unchecked)
 Auto Extend: (checked)
 Increment: 100M
 Maximum File Size: Unlimited
 There are five active buttons labelled as Back, Show SQL, OK, Cancel, and Next.

5. **The file appears in the File Name list. Deselect "Auto Extend" and click the right arrow. Note: You will set the datafile to autoextend in the Modifying a Tablespace section.**

This screenshot shows the Create Tablespace wizard - Step 2 (Add Datafiles).

Datafiles File Name: appts.dbf

Two buttons: + (add) and x (delete)

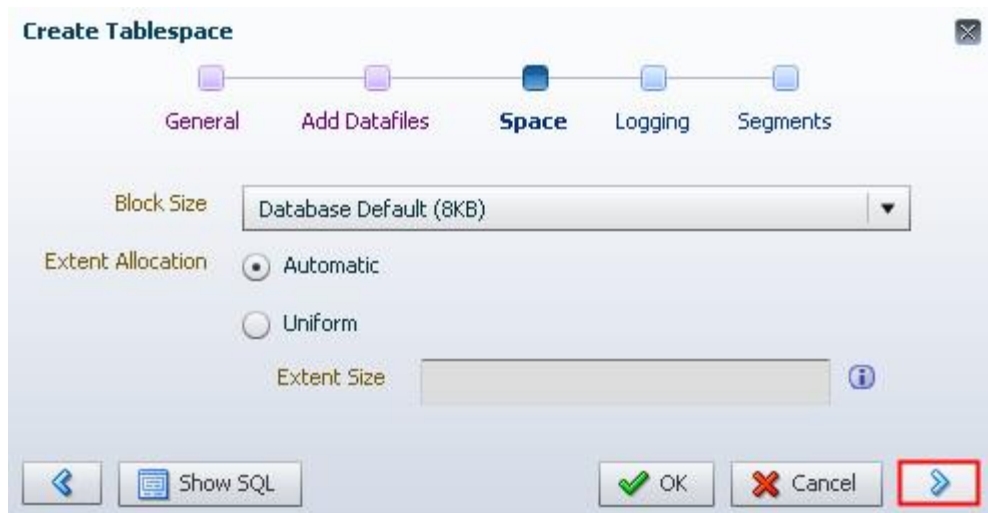
File Size: 100M

Reuse Existing File: (unchecked)

Auto Extend: (unchecked)

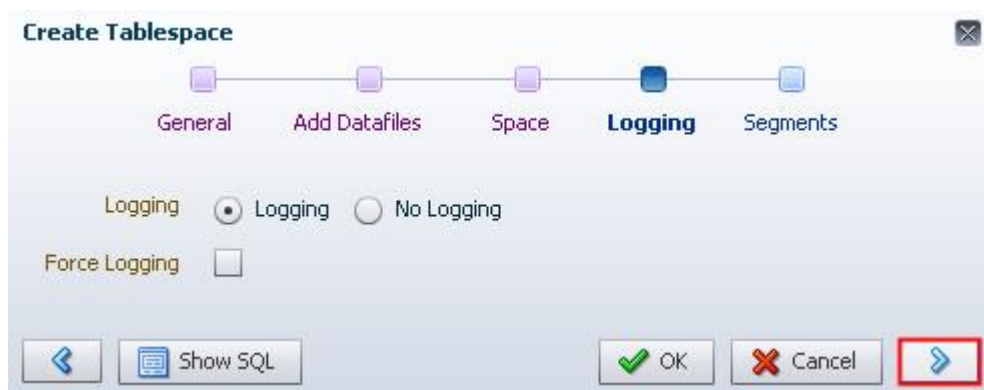
There are five active buttons labelled as Back, Show SQL, OK, Cancel, and Next. The Next button is being clicked.

6. **Accept the default block size and automatic extent allocation. Click the right arrow**



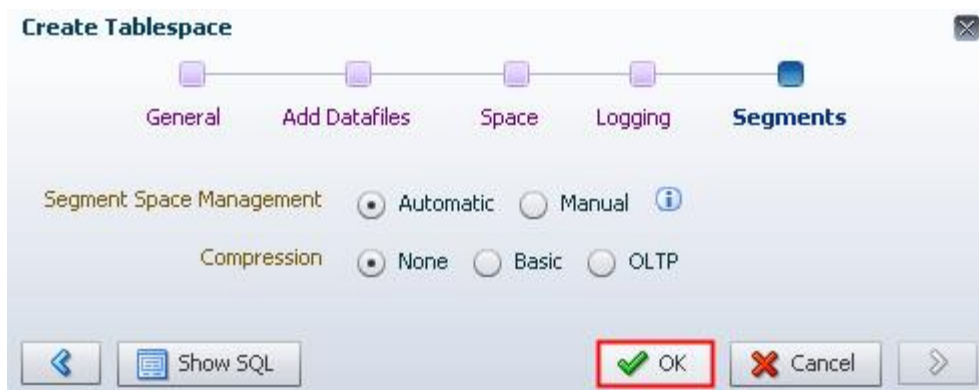
This screenshot shows the Create Tablespace wizard - Step 3 (Space). The following values are set:
 Block Size: Database Default (8KB)
 Extent Allocation: Automatic
 There are five active buttons labelled as Back, Show SQL, OK, Cancel, and Next. The Next button is being clicked.

7. Accept the default value for Logging. Click the right arrow.



This screenshot shows the Create Tablespace wizard - Step 4 (Logging). The following values are set:
 Logging: Logging
 Force Logging: (unchecked)
 There are five active buttons labelled as Back, Show SQL, OK, Cancel, and Next. The Next button is being clicked.

8. Accept the default of Automatic for Segment Space Management and None for Compression. Click OK.



This screenshot shows the Create Tablespace wizard - Step 5 (Segments). The following values are set:
 Segment Space Management: Automatic
 Compression: None
 There are four active buttons labelled as Back, Show SQL, OK, and Cancel. The OK button is being clicked.

9. Click OK on the Confirmation page.



This screenshot shows the Confirmation window with a success message that the Tablespace is created. The OK button is being clicked.

10. The new tablespace is listed on the Tablespaces page

Actions ▾ View ▾  Create  Drop  Add Datafile ☐ Permanent ☒ Tablespace											
Name	S...	F...	U...	Auto Extend	M...	Status	Type	G...	Auto Segm...	Directory	
 APPTS	10...	99...	1		100M					/scratch	
 SYSAUX	37...	27...	92.7		Unlir					/scratch	
 SYSTEM	37...	27...	92.7		Unlir					/scratch	
 TEMP	27...	62...			Unlir					/scratch	
 UNDOTBS1	62...	62...	60		Unlir					/scratch	
 USERS	100M	40M	20		Unlir					/scratch	

This screenshot shows the Tablespaces page. The APPTS tablespace is visible in the Tablespaces list.

☞ Similarly modify the table space (for detail follow the link given below)

*NOTE: visit this link to learn more activities with Oracle Enterprise Manager
Database Express (EM Express)*

[https://apexapps.oracle.com/pls/apex/f?p=44785:50:6378853118153:::50:P50_CO
URSE_ID,P50_EVENT_ID:458,6362](https://apexapps.oracle.com/pls/apex/f?p=44785:50:6378853118153:::50:P50_CO
URSE_ID,P50_EVENT_ID:458,6362)